



Advanced Agent Architectures

*AICB-P2T3 · Ngày 16 · Chương 4 —
Agent Nâng Cao · Harness & Kiến trúc*

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 4

HÃY SUY NGHĨ...

“Vì sao cùng một model, chỉ đổi harness, agent lại nhảy từ 52.8% lên 66.5%?”

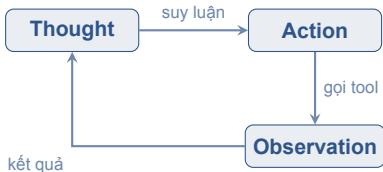
Nguồn: LangChain (2026), harness engineering trên Terminal-Bench 2.0 — xem §7, khung “Cùng một model, chỉ đổi harness”.

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Khi nào Single Agent thất bại?
2. Reflexion — Dạy Agent tự phản tỉnh
3. Bức tranh rộng hơn
4. Tích lũy kỹ năng
5. Self-Evolving Agents
6. Plan — Act — Verify
7. Harness & Middleware
8. Context Engineering cho task dài hơi
9. Orchestrator — Worker
10. Guardrails & Capability Control
11. Grounding, Citation & Hallucination Attribution
12. Đánh giá kiến trúc agent
13. Demo & Thực hành

Khi nào Single Agent thất bại?

ReAct — mạnh nhưng không biết sửa lỗi

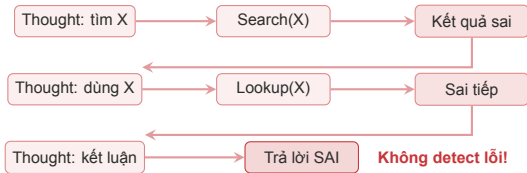


Lặp đến khi có câu trả lời

- Xen kẽ **Reasoning** (suy nghĩ) + **Acting** (hành động)
- Agent tự quyết định: gọi tool nào, khi nào dừng
- Đã học ở GD1 — nền tảng cho mọi agent pattern

Nhắc lại

ReAct = “Think before you act” — mỗi bước agent giải thích lý do trước khi hành động



3 failure modes chính:

1. **Lỗi lan tỏa:** Sai ở bước 1 → sai hết chuỗi
2. **Infinite loop:** Tool trả noise → agent lặp mãi
3. **Không backtrack:** Đi sai đường nhưng không quay lại

Lưu ý: Root cause: ReAct **không có cơ chế tự đánh giá**. Khi đi sai, không có signal nào báo “dừng lại, suy nghĩ lại”.

Nhiều tác vụ thực tế chỉ cần:

- **Retrieval** — tìm thông tin liên quan
- **Tools** — gọi API/function có sẵn
- **Structured output** — trả kết quả đúng format

Nguyên tắc: bắt đầu với single agent hoặc **workflow đơn giản** — chỉ thêm phức tạp khi có **bằng chứng** thực sự cần thiết.

Khi nào escalate?

Chỉ nâng cấp kiến trúc khi gặp **một trong ba**:

- **Tool overload:** quá nhiều tool, 1 agent không quản nổi
- **Branching logic:** prompt phải rẽ nhánh phức tạp
- **Specialist ownership:** cần agent “sở hữu” riêng một domain



“Càng nhiều agent càng tốt” là ngộ nhận — mỗi agent thêm vào là thêm latency, cost, và điểm lỗi mới. Chỉ thêm agent khi có evidence, không phải vì “nghe cool”.

Một agent ReAct review pull request — điều gì xảy ra khi bước đầu sai?

1. Agent **đọc nhầm module** trong diff — hiểu sai file nào thực sự bị đổi
2. Agent chạy **checker/search trên các file không liên quan** (dựa trên hiểu nhầm ở bước 1)
3. Agent **tổng hợp bằng chứng sai** nhưng vẫn trình bày với **độ tự tin cao**
4. ReAct **không có signal nào để dừng lại** — không ai chấm điểm bước giữa chừng

Vì sao evaluator giải quyết được?

Một evaluator có thể chấm điểm: “kết luận **có grounded** vào diff thật và test logs thật hay không?” — bắt lỗi **trước** khi agent tự tin trình bày câu trả lời sai.

35.1%

ReAct EM
trên HotpotQA

Cao

Fail rate
trên multi-hop

0

Số lần
agent tự sửa lỗi

Nguồn: *ReAct: Synergizing Reasoning and Acting in Language Models* (Yao et al.) — EM của ReAct trên HotpotQA. Hai ô còn lại là mô tả định tính, không phải số đo.

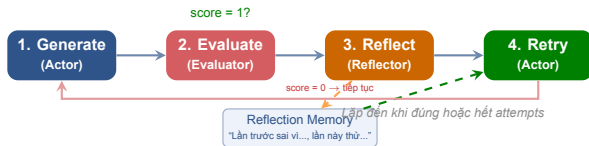
Câu hỏi then chốt

Nếu thêm cho agent khả năng **tự đánh giá** kết quả và **rút bài học** từ sai lầm thì sao?
→ Đó chính là ý tưởng của **Reflexion**.

Reflexion — Dạy Agent tự phản tỉnh

Thêm self-evaluation vào reasoning loop

Ý tưởng cốt lõi — Reflexion (Shinn et al. 2023) — Thêm 2 thành phần vào ReAct: **Evaluator** (đánh giá kết quả) và **Reflector** (rút bài học). Agent thử, đánh giá, suy ngẫm, rồi thử lại — giống cách con người học từ sai lầm. *Analogy*: như sinh viên làm bài thi — lần 1 sai → xem đáp án, hiểu tại sao sai → lần 2 làm đúng; ReAct chỉ làm 1 lần rồi nộp, Reflexion cho phép “xem lại bài” và sửa.



3 vai trò LLM

Actor: Hành động (như ReAct)
Evaluator: Chấm điểm (0 hoặc 1)
Reflector: Rút bài học từ lỗi

Điểm khác biệt vs ReAct

Dùng **text feedback** thay **gradient**.
LLM đã train trên ngôn ngữ tự nhiên
→ critique bằng text là “natural modality”

python

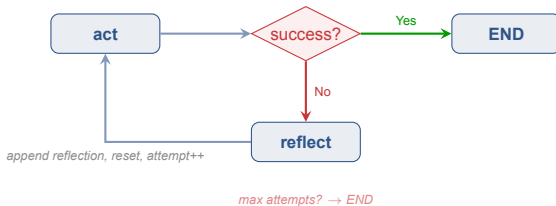
```
class ReflexionState(TypedDict):
    messages: list[BaseMessage]
    trajectory: list[str]
    reflection_memory: list[str]
    attempt_count: int
    success: bool

# Reflection prompt template
REFLECT = """
Previous attempt failed because: {error}
Lesson: {lesson}
Next strategy: {strategy}
"""
```

5 thành phần state:

1. messages: conversation hiện tại
2. trajectory: lịch sử hành động
3. reflection_memory: bài học rút ra
4. attempt_count: đếm số lần thử
5. success: đã đúng chưa?

Lưu ý: Reflection memory dùng **sliding window**: quá nhỏ → agent “quên”, quá lớn → chiếm hết context window.



Node “reflect”

1. Lấy trajectory (đã làm gì?)
2. Gọi Reflector LLM (sai ở đâu?)
3. Append reflection vào memory
4. Reset messages, tăng attempt

Termination

Dừng khi: success = True

Hoặc: attempt $\geq$ max (default 3)

Tránh infinite loop — cái mà ReAct gặp phải

91%

HumanEval pass@1
(code generation)

Tăng

Multi-hop QA &
decision-making
(mức tăng tùy task)

0

Tham số model
được cập nhật

Nguồn: Reflexion (Shinn et al., 2023) — 91% pass@1 trên HumanEval. **Chỉ con số này được trích dẫn:** mức tăng trên các task khác thay đổi theo benchmark, nên ghi định tính thay vì gán một con số duy nhất.

Tại sao hiệu quả?

Reflexion dùng **episodic memory** — agent “nhớ” bài học từ các lần thử trước *trong cùng episode*. Giống cách bạn nhớ “lần trước đã thử cách này không được, lần này thử khác”.

Evaluator là mắt xích yếu nhất của Reflexion — nếu nó chầm sai, cả vòng lặp Reflect–Retry đi sai hướng. Bốn nguyên tắc khi viết evaluator prompt:

- **Chấm theo rubric tường minh:** liệt kê tiêu chí cụ thể (đúng định dạng? đủ bước? khớp constraint nào?) thay vì hỏi chung chung “câu trả lời này tốt không?”
- **Bắt buộc trích dẫn bằng chứng** — evaluator phải chỉ ra đúng dòng/đoạn output nào khiến nó chấm như vậy, không được kết luận suông.
- **Verdict máy đọc được:** JSON {score, reason, evidence} chứ không phải văn xuôi tự do — để Reflector và code điều phối parse được.
- **Tách context với Actor:** evaluator không được thấy lý do biện minh của Actor, nếu không nó dễ bị “thuyết phục” đồng ý thay vì đánh giá độc lập.

Lưu ý: Evaluator dùng chung context với Actor = evaluator mất tính độc lập. Đây là nguồn gốc của correlated failure ở khung tiếp theo.

Lưu

Bài học rút ra: câu ngắn gọn dạng “thử X không được vì Y, lần sau thử Z” — cô đọng, hành động được, không phải tường thuật.

Chiến lược đã thất bại: tên/mô tả ngắn của hướng tiếp cận đã thử và trượt, để Reflector không đề xuất lại chính hướng đó.

Ràng buộc mới phát hiện: constraint task không nói rõ nhưng attempt trước đã lộ ra (định dạng bắt buộc, giới hạn API...)

Không lưu

Toàn bộ **lịch sử chat** thô của attempt cũ — giữ nguyên chỉ tốn token, không tăng khả năng sửa lỗi.

Tool dump đầy đủ (raw JSON, log request/response) — chỉ phần kết luận mới đáng giữ.

Diễn giải lại task — không thêm thông tin mới, chỉ lặp điều actor đã biết.

Reflexion không miễn nhiễm lỗi — 4 kiểu thất bại thường gặp khi đưa vào production:

- **Correlated failure:** Evaluator và Actor dùng chung model/prompt base → evaluator “đồng ý” với câu trả lời sai của Actor thay vì bắt lỗi độc lập.
- **Reflection không đổi chiến lược:** reflection chỉ diễn đạt lại lỗi (“câu trả lời sai vì thiếu thông tin”) mà không đưa ra hướng thử khác cụ thể → attempt 2 lặp lại attempt 1.
- **Retry không giới hạn:** thiếu max_attempts cứng → agent lặp vô hạn, đốt token/budget khi task thực sự không giải được.
- **Reflection memory mâu thuẫn nhau:** qua nhiều attempts, các bài học tích lũy lại *chối nhau* (“nên dùng cách A” rồi lại “không nên dùng cách A”) — agent bối rối hơn là học được.



Reflexion **không phải một model khác** và cũng không phải một “kiến trúc đối thủ” của ReAct. Nó là một **critic layer** — Evaluator + Reflector — được **quán quanh** vòng lặp reasoning đã có sẵn.

Nhìn theo góc harness (như đã đặt vấn đề ở đầu bài): ReAct là vòng lặp act–observe cơ bản; Reflexion thêm một **bước kiểm tra sau khi model sinh output**, trước khi output đó được chấp nhận là kết quả cuối.

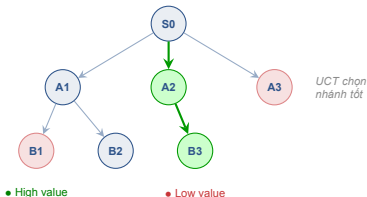
Forward-reference — §7

Ở phần Middleware & Harness Engineering (§7), chính lớp critic này sẽ được hiện thực hoá như một **after_model hook**: một điểm móc chạy sau khi model trả lời, trước khi harness quyết định bước tiếp theo. Reflexion là ví dụ đầu tiên của một pattern harness tổng quát hơn.

Bức tranh rộng hơn

LATS, Voyager và khi nào nên dùng agent phức tạp

03



LATS = search layer

Giống Reflexion là **critic layer**, tree search (ToT/LATS/MCTS) là **search layer** của harness — thêm vào khi failure mode là “**đi sai một nhánh, không quay lại được**” (failure mode thứ 3 ở §1).

LATS — MCTS + LLM: mỗi node là một trạng thái suy luận, LLM đóng vai policy + value + simulation

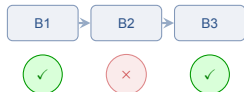
- Chính xác hơn Reflexion (92.7% vs 91%)
- **Nhưng** tốn gấp 3–5× compute
- Cần environment cho phép **undo**
- **ToT + Reflexion** → **LATS**: ToT chỉ expand cây, Reflexion chỉ reflect — LATS làm cả hai cộng MCTS search

Lưu ý: LATS chỉ đáng dùng khi task giá trị cao, cho phép rollback (code gen, game).
Nguồn: arXiv:2310.04406.

ORM: chỉ chấm đáp án cuối



PRM: chấm từng bước

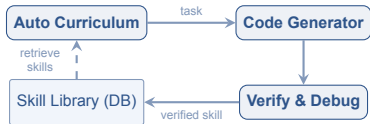


PRM bắt lỗi ở B2 ngay khi nó xảy ra — ORM chỉ biết sau khi đã đi hết B3

- **ORM** (outcome reward model): chỉ biết đáp án cuối đúng/sai — không biết bước nào đã sai
- **PRM** (process reward model): chấm **từng bước suy luận**, bắt lỗi sớm trước khi nó lan sang bước sau
- **Thinking verifier** (ThinkPRM): sinh **verification chain-of-thought** cho mỗi bước thay vì chỉ gán một điểm số rời rạc

Chi phí gán nhãn

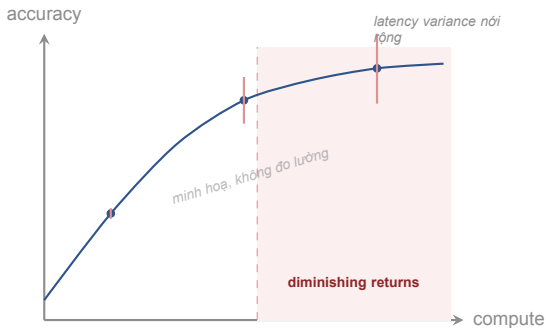
Thinking verifier đạt hiệu năng mạnh với **ít hơn nhiều bậc độ lớn** (orders of magnitude fewer) nhãn huấn luyện so với PRM discriminative.
Nguồn: ThinkPRM (2025).



- Agent tự đặt mục tiêu, viết code, lưu skill đã verified
- Skill mới xây trên skill cũ (**compound learning**)
- Tích lũy kỹ năng độc nhất nhiều hơn hẳn baseline không có skill library

Ứng dụng thực tế

Pattern Voyager áp dụng cho code generation, DevOps automation — bất kỳ domain nào agent cần tích lũy “thư viện kinh nghiệm”. Nguồn: Voyager (Wang et al., 2023); mô tả định tính, không trích dẫn con số.



Trực định tính — xu hướng minh hoạ, không phải số liệu đo được.

- Accuracy **tăng** theo test-time compute, nhưng **diminishing returns** đến rất nhanh
- Latency **variance** **nói rộng** — outlier ngày càng xa trung vị
- Chi phí hạ tầng có điểm **không còn justify được** accuracy gain nhỏ dần

Lưu ý: Hướng nghiên cứu đúng không phải “search nhiều hơn” mà là **compute-efficient reasoning**. Nguồn: arXiv:2506.04301.

1. *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models* — arXiv:2310.04406. LATS mở rộng Reflexion bằng tree expansion + Monte Carlo Tree Search; ToT, ReAct, Reflexion mỗi cái chỉ làm tree expansion **hoặc** reflection, không kết hợp cả hai với search.
2. *The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective* — arXiv:2506.04301. Agent tăng accuracy khi tăng test-time compute, nhưng diminishing returns nhanh, latency variance nói rộng, và chi phí hạ tầng đến lúc không còn justify được — hướng đi đúng là compute-efficient reasoning.
3. **ThinkPRM** (2025). Thinking verifier sinh một verification chain-of-thought cho mỗi bước suy luận, đạt hiệu năng mạnh với ít hơn nhiều bậc độ lớn (orders of magnitude fewer) nhãn huấn luyện so với discriminative PRM.

Tích lũy kỹ năng

Voyager sống tiếp năm 2026: skill layer của harness

04

Skill Library (DB) của Voyager (§3) tái xuất hiện 2026 dưới dạng **skill files**: mỗi kỹ năng một file, nạp **progressive** — chỉ khi agent chạm tới, không nhồi hết vào context từ đầu.

- **Skill files (on-demand)**: kiến thức domain chi tiết, chỉ load khi task chạm đến — giống Skill Library retrieve theo Auto Curriculum của Voyager.
- **Persistent instruction file**: đọc **một lần lúc khởi động** (agents-file, kiểu CLAUDE.md) — áp dụng luôn, không đợi task chạm tới.



Voyager là **skill layer** của harness — cấp kiến thức domain theo yêu cầu, giống Reflexion là **critic layer** (§2), tree search là **search layer** (§3).

Forward-reference — §6

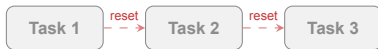
Ở §6, skill layer là một trong bốn lớp context của **deep agents** — cùng **plan file**, **filesystem-as-memory**, **subagent isolation**.

Self-Evolving Agents

Khi cải tiến sống sót qua ranh giới episode — §4 đã làm điều đó một lần

05

Cải tiến *trong* episode — §2, §3



Cải tiến *sống tiếp* sang task sau — §4



Hai câu hỏi: *cái gì* thay đổi — và thay đổi đó có **sống** qua **ranh giới episode** không?

Từ model tĩnh sang hệ sinh thái tự chủ

Survey xếp câu trả lời trên quỹ đạo **bốn giai đoạn**: **MOP** (pretraining offline) → **MOA** (thích nghi online) → **MAO** (điều phối đa agent) → **MASE** (đa agent tự tiến hoá). Nguồn: arXiv:2508.07407.

Self-evolving agent — Agent tự sửa **chính mình** (prompt, kỹ năng, bộ nhớ, trọng số, code) và **giữ lại** thay đổi sau khi task kết thúc

- **Reflexion (§2)**: sửa được trajectory, rồi quên khi episode kết thúc.
- **Tree search (§3)**: cả cây search bị bỏ đi ngay khi chọn xong nhánh.
- **Voyager (§4) thì không quên**: skill verified ghi ra Skill Library, task sau dùng lại được — **đó đã là self-evolution**.

Nấc thang	Đã gặp ở đâu trong bài	Chiều theo survey	Hôm nay
Prompt / instructions	Persistent instruction file, agents-file (§4, §7)	Environment — static knowledge	Dùng được ngay
Skills / tools	Skill Library của Voyager, skill files (§4)	Environment — modular architecture	Dùng được ngay
Memory / kinh nghiệm	Kinh nghiệm tích lũy qua nhiều phiên (Ngày 17)	Environment — dynamic experience	Đang vào production
Topology đa agent	Ai gọi ai, chia việc thế nào (§9)	Environment — agentic topology	Phần lớn còn thử công
Trọng số model	Fine-tune từ trajectory đã lọc, RL từ môi trường	Model-centric — training-based	Offline, tốn kém
Code & kiến trúc của chính agent	Agent tự sửa mã nguồn của mình (DGM, khung sau)	Co-evolution model $\leftrightarrow$ environment	Nghiên cứu

*Ba chiều phân loại và quỹ đạo MOP–MASE: A Comprehensive Survey of Self-Evolving AI Agents — arXiv:2508.07407.
Cột “Hôm nay” là **đánh giá của bài giảng**, không phải số liệu của paper.*

50.0%

SWE-bench
— từ **20.0%** trước
khi tự tiến hoá

30.7%

Polyglot
— từ **14.2%**, cùng
một vòng lặp

Transfer

Cải tiến vẫn hiệu
quả khi **đổi foundation model**

Vòng lặp: **lấy một agent từ archive** → foundation model **sinh một biến thể** (sửa chính mã nguồn của agent) → **validate bằng benchmark** → chỉ giữ lại cái chạy được. Archive kiểu Darwin giữ cả nhánh “kém hơn” nên cây agent **đa dạng**, khám phá được song song.

Kết quả đáng chú ý không phải điểm số

Phần thú vị là **cái gì** nó tự tìm ra: tool sửa code tốt hơn, cách quản lý long-context, cơ chế peer-review giữa các bước — những cải tiến **nền tảng và chuyển được**, vẫn hiệu quả khi **đổi foundation model** thay vì chỉ overfit vào một model. Nguồn: Darwin Gödel Machine (Sakana AI, 5/2025) — arXiv:2505.22954.

DGM **không tin** biến thể nào cả: **mọi** lần tự sửa code đều phải chạy qua benchmark, không qua thì bị loại — eval harness là **điều kiện tiên quyết** để được phép tiến hoá. (Nguồn: arXiv:2505.22954.)

- **Chấm tự động bằng verifier** — vòng lặp sinh hàng trăm biến thể, không chấm bằng người được.
- **Regression set bắt buộc**: biến thể mới không được phá kỹ năng cũ — đúng là định luật **Excel** (performance preservation) của survey (arXiv:2508.07407).
- **Kiểm tra tính chuyển được**: validate trên nhiều task/model để phân biệt *cải tiến thật* với *overfit* vào *một benchmark*.
- **Trace hợp lệ trước đã**: không có trace chuẩn từ middleware (§7) thì không chấm được chiều nào, nên cũng không có tín hiệu để tiến hoá.

Lưu ý: Metric yếu thì agent tiến hoá **theo metric**, không theo mục tiêu — evaluator (§2) ở đây thành **hàm mục tiêu của cả quá trình**, sai lệch bị khuếch đại qua từng thế hệ.

Forward-reference — §12

§12 (Đánh giá kiến trúc agent) là điều kiện tiên quyết của **cả section này**: benchmark map, trajectory evaluation và trace-conformance gate PASS/FAIL là bộ đo trả lời “tốt hơn hay chỉ khác đi?”.

Agent sửa được mã nguồn của chính nó thì **guardrail nằm trong mã nguồn đó cũng nằm trong tầm với** — đó là lý do định luật **đầu tiên** là **Endure**, không phải Evolve. (Nguồn: arXiv:2508.07407.)

1. Endure

Safety adaptation — biến thể phá ràng buộc an toàn thì bị loại, dù điểm có cao hơn.

2. Excel

Performance preservation — mạnh thêm ở task mới mà yếu đi ở task cũ là **regression**.

3. Evolve

Autonomous evolution — chỉ có nghĩa **khi hai điều trên đã giữ được**.

Lưu ý: Vùng agent tự sửa được và vùng chứa guardrail phải **tách rời**: allowlist, capability manifest, budget breaker nằm ở **tầng infrastructure** (§10) — tầng mà vòng lặp tiến hoá không có quyền ghi.



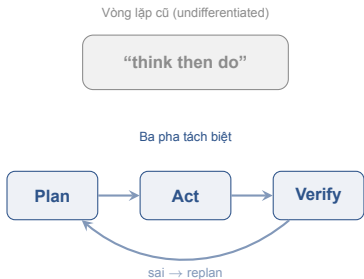
Self-evolution là **một lớp nữa** của harness — lớp **persistence**. Như critic (§2), search (§3), skill (§4): chỉ thêm **khi có bằng chứng** — ở đây bằng chứng đắt hơn hẳn: cả một eval harness (§12).

1. **Darwin Gödel Machine** (Sakana AI, 5/2025) — arXiv:2505.22954. Hệ tự cải tiến lặp lại việc **sửa chính mã nguồn của mình** và **kiểm chứng thực nghiệm** từng thay đổi trên coding benchmark; giữ một **archive** agent theo kiểu Darwin (lấy mẫu một agent, dùng foundation model sinh biến thể, giữ lại cái validate được) nên sinh ra một cây agent đa dạng, khám phá song song. SWE-bench 20.0% → 50.0%; Polyglot 14.2% → 30.7%. Các cải tiến nó tự tìm ra (tool sửa code, quản lý long-context, cơ chế peer-review) mang tính nền tảng và **chuyển được sang foundation model khác**.
2. *A Comprehensive Survey of Self-Evolving AI Agents* — arXiv:2508.07407. Khung quy chiếu chuẩn: quỹ đạo bốn giai đoạn **MOP** → **MOA** → **MAO** → **MASE** (từ model tĩnh do người cấu hình sang hệ sinh thái đa agent tự chủ); ba chiều tiến hoá **model-centric** (inference-based: parallel sampling, sequential self-correction, structured reasoning — và training-based: synthesis-driven offline, exploration-driven online), **environment-centric** (static knowledge, dynamic experience, modular architecture, agentic topology) và **co-evolution** model–environment; ba định luật **Endure** (safety adaptation) — **Excel** (performance preservation) — **Evolve** (autonomous evolution).

Plan — Act — Verify

Tách ba pha thay vì một vòng lặp “think then do”

06

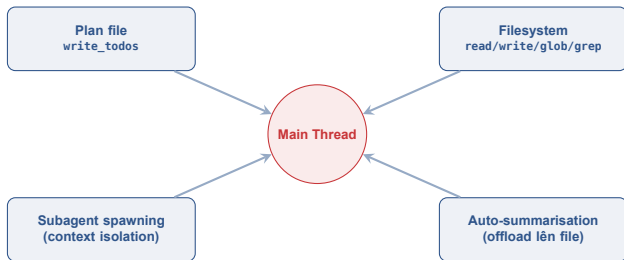


Ba tín hiệu, không phải một

Plan thất bại (mục tiêu sai), Act thất bại (tool lỗi), Verify thất bại (không đạt tiêu chí) là **ba loại lỗi khác nhau** — trộn chung vào một prompt xóa mất sự phân biệt này.

- Vòng lặp “think then do” undifferentiated dùng **cùng một prompt** cho mọi việc — suy nghĩ, hành động, kiểm tra trộn vào một lượt gọi model.
- Tách **Plan / Act / Verify** cho mỗi pha một **prompt riêng**, một **budget compute riêng**, một **tín hiệu thất bại riêng**.
- Kết quả: pha nào hỏng, biết **ngay pha đó** — không phải đoán lại từ một transcript lẫn lộn cả ba việc.

Lưu ý: Tách pha không miễn phí: mỗi pha là một lượt gọi model riêng — đổi latency lấy sự ổn định và khả năng debug.



Không thành phần nào tự làm agent “deep” — chính **sự kết hợp** bốn thành phần mới tạo khả năng xử lý task dài hơi.

- **Plan file:** artifact ghi bằng tool `write_todos` — không phải text trong prompt.
- **Filesystem = external memory:** `read/write/edit_file`, `ls/glob/grep` thay context window lưu trạng thái.
- **Subagent spawning:** việc phụ giao subagent riêng — main thread giữ **sạch**.
- **Auto-summarisation:** output tool lớn offload ra file.

Bốn lớp context

skills (§4) → memory (agents-file, lúc khởi động) → summarisation/offloading → prompt caching. Nguồn: LangChain Deep Agents runtime (2026).

Plan trong context — Một đoạn text trong prompt, **viết lại mỗi turn** để phản ánh tiến độ mới nhất

- Bị **drift**: mỗi lần viết lại, chi tiết cũ có thể bị bỏ sót/diễn giải khác đi.
- Không **diff** được giữa hai turn — không rõ đã đổi chỗ nào.
- Mất trắng nếu context bị **compact** — không có bản gốc để khôi phục.

Plan file trên đĩa — Một file thật — `write_todos` cập nhật bằng **edit**, không viết lại toàn bộ

- **Addressable**: mỗi mục có vị trí cố định, tham chiếu được bằng tên/dòng.
- **Diffable**: so sánh hai thời điểm để biết chính xác đã đổi gì.
- **Sống sót qua compaction**: nằm ngoài context window, nên context không đụng tới.

Forward-reference — §8

Ở phần Context Engineering (§8), **compaction** — nén context khi conversation dài ra — là lý do khiến plan-on-disk trở thành yêu cầu bắt buộc, không chỉ lựa chọn phong cách, cho task dài hơi.

Harness & Middleware

Sáu hook — nơi mọi lớp ở §2–§6 trở thành code

07

52.8% → 66.5%

Terminal-Bench 2.0
(coding agent)

top 30+ → #5

Thứ hạng: từ
ngoài top 30
lên hạng 5

0

Thay đổi ở model
(cùng model,
cùng weights)



Toàn bộ mức tăng đến từ **harness engineering** — lớp code bao quanh model — chứ không phải từ một model mạnh hơn. Hiệu năng của agent **không nằm hết trong model**.

Nguồn

LangChain (2026), harness engineering trên **Terminal-Bench 2.0** — **con số duy nhất** của §7.

Model là hằng số bạn thuê. **Harness** là phần bạn viết — và nó chỉ có **ba núm xoay**:

1. **System prompt** — Model là ai, cư xử thế nào

- Rẻ nhất để thử, dễ nhất để lạm dụng.
- Sửa được *hành vi*, không sửa được *cơ chế*.
- **Cấu hình tĩnh**: viết một lần, áp cho mọi lượt.

2. **Tools** — Agent **chạm** được vào cái gì

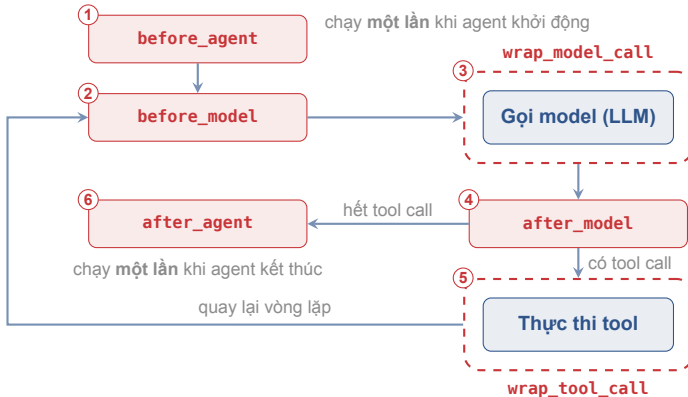
- Mỗi tool là một khả năng — và một bề mặt rủi ro.
- Thêm tool: mạnh hơn **và** khó chọn đúng hơn.
- Cũng **tĩnh**: khai báo lúc dựng agent.

3. **Middleware** — Code chạy **quanh** từng bước vòng lặp

- Núm duy nhất **lập trình được**: có điều kiện, có state.
- Thấy thứ model không thấy: lịch sử, ngân sách, mẫu lặp.
- Nơi **§2–§6** biến thành code thật.



Prompt và tools chỉ **mô tả** agent; middleware **can thiệp** lúc agent đang chạy — đọc được state, chặn được hành động, sửa được context, dừng được vòng lặp.



critic layer (§2) = after_model · **skill layer** (§4) = nạp skill file ở before_model · **search layer** (§3) = điều khiển quanh wrap_model_call · **deep-agent context** (§6) = before_model + wrap_tool_call. Không cái nào là “kiến trúc đối thủ”.

wrap_tool_call chạy cả **trước** lẫn **sau** lời gọi tool — nên thấy cả đầu vào lẫn đầu ra.

Ba việc chỉ hook này làm được:

- **Gating.** Model *đề nghị*, harness *cho phép*: chặn, đổi tham số, hoặc trả lỗi — trước mọi side effect.
- **Injecting context.** Tự điền tenant/user, giới hạn scope, gắn request-id — thứ model **không cần biết**.
- **Intercepting results.** Redact PII, cắt output không lờ, hoặc **offload ra file** rồi chỉ trả đường dẫn (§6).

Vì sao không đặt ở prompt? — “Đừng gọi tool xóa khi chưa xác nhận” trong system prompt là **lời đề nghị**; cùng luật đó trong wrap_tool_call là **bất biến**.

Dynamic tool selection

Tập tool **đổi được theo state**: pha Plan thấy tool khác pha Act (§6) — giảm **tool overload** (§1).

Forward-reference — §10

Guardrails (§10) gắn vào **đúng hook này**: policy engine, allowlist, human-in-the-loop approval.

before_model chạy **ngay trước mỗi lượt gọi model**, sau khi state đã cập nhật — điểm duy nhất bạn kiểm soát **toàn bộ nội dung** model sắp đọc.

1. Context summarisation — nén thứ đã dài ra

- Lịch sử hội thoại và tool output tích lũy mỗi vòng; đến ngưỡng, hook tóm tắt phần cũ và thay bằng bản nén.
- **Nén cái gì** là quyết định kỹ thuật, không phải của model: giữ plan file, kết quả verify, ràng buộc gốc — nén phần transcript trung gian.
- Thiếu hook này, task dài hơi kết thúc bằng “đầy context” chứ không phải “xong việc”.

2. Context injection — thêm thứ đang thiếu

- Nạp **skill file** liên quan tới bước hiện tại (§4) — progressive, chỉ khi chạm tới.
- Chèn lại **plan file** và trạng thái todo (§6) để model không trôi khỏi mục tiêu sau nhiều vòng.
- Chèn **bài học** từ lượt trước — chính là reflection memory của §2, đọc từ store thay vì để nằm trong transcript.

Forward-reference — §8

Context engineering (§8) trả lời *nén cái gì, giữ cái gì, ở ngưỡng nào*. Ở đây chỉ cần nhớ: dù chiến lược là gì, **nơi cắm nó vào** luôn là before_model.

Ở §2 đã hứa: **critic layer** của Reflexion sẽ là một **after_model hook**. Đây là lúc trả lời.

Ảnh xạ một-một với §2:

- **Evaluator** → **after_model**: chấm output vừa sinh.
- **Reflector** → cũng ở đó: sinh bài học, ghi vào reflection memory.
- **Vòng thử lại** → hook **không chấp nhận** output, đẩy state quay lại vòng lặp — model chạy lần nữa với bài học vừa ghi.

Reflexion vì thế **không phải kiến trúc riêng** — nó là **một cách hiện thực** của **after_model**.

Còn **wrap_model_call** thì sao?

Nó bọc **bản thân lời gọi** (retry, fallback, latency/cost); **after_model** nói về **nội dung**.

Cùng hook, nhiều luật khác — Critic đã là hook thì luật khác cắm vào **cùng chỗ**, không đụng tới agent

- **Validate output**: sai schema → bắt sinh lại.
- **Redact**: cắt PII trước khi output vào log.
- **Mẫu bất thường**: model lặp lại chính mình — trang sau.
- **Đếm ngân sách**: trừ token/lượt gọi của lượt vừa xong.

Bốn hook trước chạy **mỗi vòng lặp**; hai hook này chạy **một lần** — mở và đóng toàn bộ phiên chạy.

Budget accounting — cần **điểm mở** và **điểm chốt**

- before_agent: khởi tạo hạn mức — token, tool call, vòng lặp, thời gian.
- Mỗi vòng, các hook khác **trừ dần** vào hạn mức đó.
- after_agent: chốt sổ — tổng token, tổng cost, đã chạm trần nào chưa.
- Thiếu cặp hook này, “task này tốn bao nhiêu?” là câu hỏi **không trả lời được**.

Trace emission — một lần chạy = **một trace**

- before_agent: mở root span, gắn run-id, ghi input và cấu hình.
- Mỗi lượt model và mỗi lượt tool là một span con treo dưới root span đó.
- after_agent: đóng span, gắn kết quả và trạng thái kết thúc.
- Nhờ vậy mới **diff** được hai lần chạy: cùng task, khác harness.



Con số ở đầu §7 chỉ **đo được** vì mỗi lần chạy đều có ngân sách và trace. Thiếu hai hook này, bạn vẫn sửa được harness — nhưng **không biết** mình đã cải thiện hay làm tệ đi.

§1 nêu ba failure mode của ReAct. Failure mode thứ hai — **infinite loop**: tool trả noise, agent lặp mãi — đến giờ vẫn chưa được giải quyết. Reflexion (§2) chấm *chất lượng*, không chấm *sự lặp lại*.

Vì sao model không tự thấy?

Từ bên trong context, vòng lặp thứ N **trông y hệt** vòng thứ nhất. Sự lặp lại là **thuộc tính của lịch sử** — mà lịch sử đó có thể đã bị nén mất (§6).

Middleware thì thấy, vì nó giữ state qua các vòng:

- cùng tool, cùng tham số, lặp quá N lần; hoặc hành động quay về đúng trạng thái cũ;
- plan file (§6) **không đổi** nhiều vòng liên.

Bốn cách phá vòng lặp — theo thứ tự leo thang:

1. **Cảnh báo vào context** (`before_model`): “đã thử cách này N lần, không tiến triển”.
2. **Chặn hành động lặp** (`wrap_tool_call`): từ chối đúng lời gọi đang lặp, buộc agent đổi cách.
3. **Ép đổi chiến lược** (`after_model`): không chấp nhận output — critic layer với tiêu chí “lặp lại”.
4. **Dừng và bàn giao** (`after_agent`): hết ngân sách → dừng **có kiểm soát**, trả trace.

§1 Infinite loop là lỗi ở **tầng vòng lặp** — chỉ sửa được ở nơi **nhìn thấy** vòng lặp: **middleware**.

1. **LangChain (2026)** — *Harness engineering trên Terminal-Bench 2.0*. Một coding agent đi từ **52.8%** lên **66.5%**, từ ngoài **top 30** lên **hạng 5**, **không đổi model** — toàn bộ mức tăng đến từ harness engineering. Đây là nguồn của **con số duy nhất** xuất hiện trong §7. (Không có số arXiv cho công bố này — đừng trích dẫn kèm một mã arXiv nào.)
2. **LangChain agent middleware (2026)** — sáu hook chặn và sửa được vòng chạy của agent, theo đúng thứ tự: `before_agent`, `before_model`, `wrap_model_call`, `after_model`, `wrap_tool_call`, `after_agent`. Các ứng dụng đã được tài liệu hoá: PII redaction, dynamic tool selection, context summarisation, model retries, và phát hiện các mẫu có vấn đề như **doom loop**. Riêng `wrap_tool_call` bọc quanh việc thực thi tool: chèn context, chặn kết quả trả về, và quyết định tool nào thực sự được chạy. (Không có số arXiv.)
3. **Ba núm xoay của harness** — system prompt, tools, middleware. Hai núm đầu là cấu hình tĩnh; núm thứ ba là núm duy nhất lập trình được, và là nơi mọi lớp ở §2–§6 (critic, search, skill, context của deep agent) được hiện thực hoá.
4. **Từ vựng dùng chung với lab** — scaffold của **lab Agent Arena** có sẵn stub cho **đúng sáu tên hook** ở mục 2, viết y hệt cách viết ở trên (kể cả dấu gạch dưới). Đổi tên, đổi thứ tự, hay viết sai một ký tự = lab không chạy. Khi làm lab, mở lại slide sơ đồ sáu hook để đối chiếu.
5. **Đọc thêm** — Anthropic, *Building Effective Agents*: cùng lập luận “bắt đầu đơn giản, chỉ thêm phức tạp khi có bằng chứng” đã dùng ở §1, nhìn từ phía thiết kế hệ thống thay vì phía hook.

Context Engineering cho task dài hơi

Nén context trước khi nó tràn — cắm vào đúng hook `before_model`

+29%

Context editing
(một mình)

+39%

Context editing
+ memory tool

84%

Giảm token dùng
(eval web-
search 100 lượt)

Chốt

84% không chỉ là tiết kiệm chi phí — ở eval 100 lượt, nó là ranh giới giữa **chạy xong** và **hết context giữa chừng**. Đây là vấn đề *khả năng hoàn thành task*, không chỉ vấn đề ngân sách.

Nguồn

Anthropic, tài liệu hướng dẫn context engineering (2026) — nội bộ evaluation. (Không có số arXiv cho công bố này.)

Sai lầm phổ biến nhất: đợi context **đầy** rồi mới nén. Lúc đó model đã bắt đầu mất chất lượng vì context quá dài — nén muộn không cứu được lượt gọi vừa hỏng.

Ngưỡng kích hoạt

- Workload đơn giản: nén ở khoảng **5–20k token**.
- Workload phức tạp (nhiều tool, nhiều bước): nén ở khoảng **50–100k token**.
- Cắm vào `before_model` (§7) — hook chạy ngay trước mỗi lượt gọi, đúng chỗ để quyết định model thấy gì.

Vì sao không phải truncation —
Compaction dùng **một LLM** để viết một bản tóm tắt toàn diện (holistic summary)

- Giữ thông tin theo **độ liên quan ngữ nghĩa** (semantic relevance) — ví dụ minh họa (không phải số đo): một chi tiết quan trọng từ *đầu* hội thoại vẫn giữ được dù đã qua rất nhiều lượt.
- Truncation cắt theo **vị trí** (position) — luôn mất phần cũ nhất, kể cả khi phần đó vẫn còn quan trọng.

Compaction

- Một **LLM viết lại** toàn bộ lịch sử thành bản tóm tắt holistic.
- **Lossy**: chi tiết bị paraphrase, có thể trôi nghĩa.
- **Tốn một lượt gọi model** — có latency và chi phí riêng.
- Giữ được thông tin theo **độ liên quan**, không theo vị trí.

Structured eviction

- **Xoá theo luật** cố định — vd. tool result cũ hơn N lượt, hoặc output đã offload ra file (§6).
- **Rẻ**: không gọi model, chỉ áp một điều kiện.
- **Dự đoán được**: biết trước chính xác cái gì bị bỏ, khi nào.
- Không cứu được thông tin **rải rác** giữa lịch sử — chỉ cắt theo vị trí/luật.

Ngày 16 nén context trong **MỘT phiên đang chạy** — nhưng agent có nhớ gì khi phiên đó kết thúc không? Đó là câu hỏi của một ngày khác.

Ngày 16 có

- Compaction & eviction trong **một context window đang chạy**: ngưỡng kích hoạt, cơ chế tóm tắt holistic theo semantic relevance.
- Nơi cắm vào harness: hook `before_model` (§7).

Ngày 16 KHÔNG có → Ngày 17

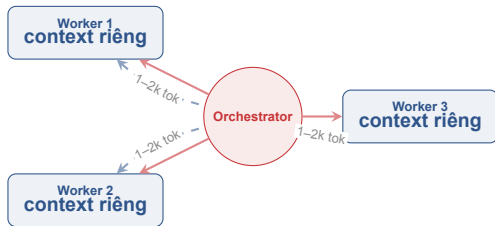
- Bộ nhớ sống sót qua session (persistent cross-session memory).
- Memory store (nơi lưu bộ nhớ dài hạn).
- Retrieval-backed memory (truy hồi bộ nhớ theo truy vấn).

Nguồn §8: **Anthropic (2026)**, tài liệu hướng dẫn context engineering — context editing một mình **+29%**, kèm memory tool **+39%**, và giảm **84%** token trên eval tìm kiếm web 100 lượt (các con số ở khung “Compaction thắng bao nhiêu?”). Không có số arXiv cho công bố này — đừng trích dẫn kèm một mã arXiv nào. Forward-reference của §7 đã đóng: ngưỡng 5–20k / 50–100k token là tham số cấu hình của `before_model`.

Orchestrator — Worker

Phân rã việc ra nhiều context riêng biệt

09



nét liền = giao việc nét đứt = tóm tắt trả về

- Orchestrator **tách task** thành các phần việc con, giao mỗi phần cho một subagent.
- Mỗi worker chạy với **context window cô lập** — không thấy transcript của worker khác hay của orchestrator.
- Worker **không trả nguyên transcript** — chỉ một bản tóm tắt cô đọng, khoảng **1.000–2.000 token**.
- Đây chính là **subagent spawning** ở §6 (deep agents), nhìn ở quy mô một hệ nhiều agent.

Nguồn

Anthropic, hệ thống multi-agent research (2026). (Không có số arXiv.)

Khi CHƯA cần multi-agent

- Một agent **đơn** với tool tốt vẫn thắng — ít moving part hơn, dễ debug hơn.
- Task vừa đủ trong **một context window** — không cần cô lập.
- Chi phí bị **chi phí điều phối** (coordination) chi phối: orchestrator + N worker = $N+1$ lượt gọi model, cho một việc một agent làm được.
- Debug khó hơn: lỗi có thể nằm ở *điều phối*, không nằm ở bất kỳ worker nào.

Khi multi-agent bắt đầu đáng tiền

- **Quá tải tool**: một agent phải chọn giữa hàng chục tool — tách theo chuyên môn giảm nhiều lựa chọn.
- Subtask **song song thật sự** — không phụ thuộc kết quả của nhau, chạy đồng thời rút ngắn latency tổng.
- **Chuyên môn hoá theo chủ sở hữu**: mỗi worker sở hữu một domain, một bộ tool riêng, một tiêu chí đúng/sai riêng.
- **Cô lập context là mục đích**, không phải tác dụng phụ — worker ô nhiễm không lan sang orchestrator hay worker khác.

Orchestrator đã biết **phân việc** và **cô lập context** — nhưng các agent nói chuyện với nhau thế nào? Đó là câu hỏi của một ngày khác.

Ngày 16 có

- **Anatomy**: ai gọi ai, context cô lập ra sao, tóm tắt trả về bao nhiêu token.
- **Kinh tế học**: khi nào multi-agent đáng tiền, khi nào không.

Ngày 16 KHÔNG có → Ngày 20

- Giao thức giao tiếp giữa các agent (protocol).
- Các mẫu handoff (handoff pattern) giữa orchestrator và worker.
- A2A (Agent-to-Agent) — chuẩn giao tiếp liên-agent.

Guardrails & Capability Control

§7 đã hứa: guardrails gắn vào `wrap_tool_call` — đến lúc trả lời tại sao

Prompt injection là **LLM01** — hạng mục **đứng đầu** OWASP Top 10 for LLM Applications ở cả bản **2023** lẫn bản **2025**. Agentic AI cộng thêm kết nối tool/MCP mở ra bề mặt tấn công mới. (Nguồn: OWASP Top 10 for LLM Applications.)

- **Tool poisoning** — metadata hoặc mô tả tool bị chỉnh sửa để dụ agent gọi sai.
- **Credential theft qua tool output** — output tool trả về chứa lệnh ẩn đánh cắp secret trong context.
- **Indirect injection qua nội dung retrieved** — tài liệu/RAG chunk mang chỉ thị giả trang bị như dữ liệu thường.

Lưu ý: Một nghiên cứu 2026 cho thấy **chỉ một số ít tài liệu bị chỉnh sửa** (crafted) có thể chi phối phần lớn câu trả lời của một hệ RAG. Đây là kết quả của **một nghiên cứu đơn lẻ**, không phải hằng số đã được xác lập — đọc như tín hiệu cảnh báo, không phải con số để trích dẫn nguyên văn.

- ✓ **Validate input** trước khi model nhìn thấy nó — lọc chỉ thị lạ ở tầng vào
- ✓ **Validate output** trước khi nó kích hoạt hành động — chặn output độc trước khi thực thi
- ✓ **Enforce permission qua capability manifest** tại tầng tool-call — đây chính là hook `wrap_tool_call` (§7)
- ✓ **Gate hành động không thể hoàn tác** bằng một blocking pause chờ con người duyệt
- ✓ **Giới hạn budget & action rate** như circuit breaker cứng — chặn cứng, không phải gợi ý
- Một rule chỉ tồn tại trong system prompt **không phải** một lớp phòng thủ

Model có thể bị thuyết phục bỏ qua một câu instruction trong prompt — nhưng không thể bỏ qua một allowlist chặn cứng ở tầng infrastructure.

- Mọi rule enforcement chỉ sống **trong prompt** phải được thay bằng một control ở **tầng infrastructure** — không có ngoại lệ.
- Mọi tool có thể gọi cần đủ ba thứ: **scope** định nghĩa rõ, **allowlist entry**, và **reversibility classification** (có rollback được không).

Khung chấm điểm lab — Safety

Dimension **Safety** của injection_guard chấm hai điều: **injection resistance** (agent có làm theo chỉ thị giả trong tool output/retrieved content không?) và **correct abstention** (agent có từ chối hành động ngoài scope thay vì cứ thực thi không?).

Ngày 16 xây **khái niệm** — threat model, năm lớp phòng thủ, ranh giới prompt-vs-infrastructure. Việc **triển khai** các lớp đó là câu hỏi của những ngày khác.

Ngày 16 có

- Threat model: injection, tool poisoning, RAG poisoning.
- Năm lớp defence-in-depth, gắn vào `wrap_tool_call`.
- Ranh giới prompt rule vs infrastructure control.

Ngày 16 KHÔNG có

- Ngày 24: guardrail cụ thể & LLM-as-judge để chấm guardrail.
- Ngày 25: circuit breaker thực thi & reliability engineering.
- Ngày 26: MCP — capability manifest thật cho hệ tool.

Grounding, Citation & Hallucination Attribution

Agent trích dẫn sai — lỗi nằm ở đâu, và ai bắt được nó?

41.1%

Step-localisation
accuracy của
model **tốt nhất**

11.6%

Tool-use hallucination
— nhóm khó nhất

693

Trajectory
được gán nhãn
qua 5 domain

Nguồn

AgentHallu (arXiv:2601.06818) — benchmark 693 agent trajectory qua 5 domain, taxonomy & gán nhãn đa cấp cho hallucination attribution. Bài toán khó **ngay cả với model top-tier**: 41.1% là điểm số cao nhất đạt được, không phải điểm trung bình.

Unsupported claim

Câu trả lời **không có nguồn** nào đi kèm — claim trôi nổi, không trích dẫn gì cả.

Misattributed claim

Nguồn được trích dẫn **có thật**, nhưng **không hề chứa** nội dung hỗ trợ claim đó.

Fabricated source

Nguồn được trích dẫn **không tồn tại** — agent bịa cả citation.

Vì sao phân biệt ba loại này quan trọng

Mỗi defence bắt một loại khác nhau: kiểm tra “có citation không” chỉ bắt *unsupported*; phải **đọc lại nguồn** mới bắt được *misattributed*; phải **xác minh nguồn tồn tại** mới bắt được *fabricated*. Đây chính là các bẫy corpus mà lab dùng để chấm điểm.

1. **Parse claim thành đơn vị kiểm được:** tách câu trả lời thành các claim rời rạc, mỗi claim kiểm được độc lập.
2. **Verify từng claim đối chiếu với đúng đoạn nguồn được trích,** chứ **không** chỉ kiểm tra citation có tồn tại hay không — một citation thật vẫn có thể không hỗ trợ claim (misattributed).
3. **Abstain khi thiếu bằng chứng:** agent phải trả lời “insufficient evidence” thay vì confabulate khi không tìm được nguồn hỗ trợ.

Lưu ý: citation_checker của lab chấm **Grounding** bằng citation precision/recall (claim có nguồn đúng không) cộng **hallucination penalty** (trừ điểm nặng cho unsupported/misattributed/fabricated). Abstain đúng lúc được thưởng điểm — im lặng khi thiếu bằng chứng tốt hơn là bịa.

Đánh giá kiến trúc agent

Benchmark nào đo gì — và ma trận quyết định của cả bài

12

Benchmark	Đo gì
Terminal-Bench 2.0	Agent tự động trong môi trường command-line thực tế — đọc file, chạy lệnh, sửa code, debug; chấm bằng hidden verifier tests . 89 task, mỗi agent thử 5 lần .
GAIA	466 task đòi hỏi multi-step web browsing, parse file và tool use; chấm đối chiếu ground-truth answer.
τ^2 -bench (tau2-bench)	Tool-agent-user interaction trong domain doanh nghiệp: user do LLM mô phỏng , agent gọi tool API domain-specific, phải tuân theo policy document .
OSWorld	Computer use trên desktop thật .
SWE-bench Verified	Sửa bug GitHub thật .
METR HCAST / time horizons	Task dài nhất agent hoàn thành được với xác suất 50% .

Nguồn: Terminal-Bench 2.0 (Laude Institute); GAIA (Meta FAIR/Hugging Face); τ^2 -bench (Sierra); OSWorld; SWE-bench Verified (OpenAI-curated subset); METR autonomy time-horizon evaluations.

Final-answer accuracy chỉ nói agent đúng hay sai ở đích — nó **che giấu** agent đã đi lệch ở bước nào trên đường đi.

- Trajectory evaluation chấm **con đường**: agent có chọn **đúng tool** không, có **tự phục hồi** sau lỗi không, có ở trong **ngân sách** (budget bước/token) không.
- Hai agent cùng ra đáp án đúng có thể có trajectory rất khác nhau — một cái đi thẳng, một cái lạc đường rồi may mắn tìm lại được; accuracy không phân biệt được.
- Với agent nhiều bước (Reflexion, LATS, orchestrator–worker), trajectory là nơi mọi lớp harness ở §2–§9 để lại dấu vết.

Forward-reference — Lab 16 “Agent Arena”

Lab 16 chấm **ba chiều** trên một corpus tài liệu thù địch — **Grounding** (citation precision/recall, phạt hallucination; §11, lớp citation_checker), **Safety** (kháng injection + abstain đúng lúc; §10, lớp injection_guard), **Efficiency** (token cost + latency so với ngân sách; §3, lớp budget_policy) — và trên cả ba là một **trace-conformance gate PASS/FAIL**: không sinh được trace hợp lệ thì không chấm được chiều nào. Đó là lý do middleware (§7) là bắt buộc.

Layer	Khi nào dùng	Chi phí	Dấu hiệu nên dừng lại
ReAct thuần	Task đơn giản, ít bước, không cần tự sửa	\$	Sai ngay bước đầu, không tự sửa
+ critic (§2)	Multi-step, cần tự sửa lỗi (after_model)	\$\$	Lặp self-reflect không thoát (degeneration-of-thought)
+ search (§3)	High-stakes, cho phép rollback	\$\$\$\$\$	Compute vượt giá trị, hoặc không undo được
+ skills (§4)	Lặp domain, cần tích lũy kinh nghiệm	\$\$\$	Library phình to, ít skill tái dùng (skill rot)
+ plan file (§6)	Task dài hơi, cần checkpoint & resume	\$\$\$	Plan file rewrite toàn bộ thay vì patch
+ subagents (§9)	Chia được sub-task độc lập, cần song song	\$\$\$\$	Orchestrator tốn token điều phối hơn worker làm việc

Đọc: “AI Agents That Matter” (2024) — nhiều bài toán không cần agent, đừng over-engineer.

Demo & Thực hành

Xem Reflexion hoạt động thực tế

13

LIVE DEMO

1. Cùng câu hỏi 2-hop, chạy cả hai agent với LangSmith tracing
2. ReAct: sai entity ở hop 1, lỗi lan tỏa, trả lời sai
3. Reflexion: attempt 1 sai → Evaluator cho score=0 → Reflector sinh bài học → attempt 2 đúng
4. So sánh: trace, accuracy, cost/query

LAB #16

Mục tiêu: Implement Reflexion agent từ scratch với LangGraph

Deliverable: Reflexion agent repo + benchmark report (EM comparison, cost analysis, failure categorization)

Thời gian: 2 giờ

1. **Build state machine:** nodes = [act, evaluate, reflect, terminate], edges có conditional routing
2. **Build Evaluator:** LLM-as-Judge, prompt yêu cầu score 0–1 + reason, parse output Pydantic
3. **Add reflection memory:** mỗi entry = (attempt_id, failure_reason, lesson, strategy_next)
4. **Benchmark:** Chạy Reflexion vs ReAct trên 20 câu HotpotQA — đo EM, attempts, token cost

Deliverable

GitHub repo + benchmark report: bảng so sánh EM, phân tích cost, phân loại failure modes

Những ý chính cần nhớ trước khi sang bài tiếp theo

1

Single agent thất bại theo **ba kiểu có tên** (§1): **lỗi lan tỏa**, **infinite loop**, **không backtrack**.
Chọn kiến trúc theo failure mode đang gặp.

2

Mọi kiến trúc có tên trong bài là **một lớp của harness**, không phải sản phẩm đối thủ: **critic** (§2) · **search** (§3) · **skill** (§4) · **plan file** (§6) · **subagent** (§9) — cắm vào **sáu hook** của §7.

3

Mỗi lớp đều **tốn token và latency**. Thêm lớp **khi có bằng chứng** (failure mode đo được, trace, ngân sách) — không thêm mặc định; dùng ma trận §12.

4

Các lớp còn lại học ngày khác: **memory** qua session (Ngày 17) · **giao thức multi-agent** (Ngày 20) · **guardrails** triển khai (Ngày 24) · **MCP** (Ngày 26).

Bài tiếp theo

Ngày 17: Memory Systems for Agents

“§8 chỉ nén context trong một phiên — vậy agent nhớ gì sau khi phiên đó kết thúc?”

Bài tập về nhà

- Ôn lại sơ đồ **sáu hook** (§7) — lab dùng đúng sáu tên đó
- Đọc: Anthropic, *Building Effective Agents*
- Chuẩn bị: một failure mode thật bạn từng gặp → lớp nào ở §2–§9 sẽ sửa nó?

Hỏi & Đáp

Bạn đang định thêm một lớp harness — bằng chứng nào cho thấy nó đáng chi phí token và latency?



Cảm ơn!

AICB-P2T3 · Ngày 16 · Advanced Agent Architectures

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn